COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	10/15/2026	**DATE SUBMITTED:**	10/20/2026
NAME:	Cabug, Jeffordniel H. Salcedo, Lance Cristopher S.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

HANDS-ON ACTIVITY NO. 13.1 Parallel Algorithms and Multithreading

1. PROCEDURE OUTPUT

Code:

ILOA.cpp > print(int, const std::string &)

```cpp
#include <iostream>
#include <thread>

void print(int n, const std::string &str){
    std::cout << "Printing Integer: "<< n << std::endl;
    std::cout << "Printing String: "<< str << std::endl;
}

int main() {
    std::thread t1(print, 10, "T.I.P.");
    t1.join();

    return 0;
}
```

Table 13-1. Simple One-Threaded Code

```cpp
#include <iostream>
#include <string>
#include <thread>
#include <vector>

void print(int n, const std::string &str) {
    std::string msg = std::to_string(n) + " : " + str;
    std::cout << msg << std::endl;
}

int main() {
    std::vector<std::string> s = {
        "T.I.P.",
        "Competent",
        "Computer",
        "Engineers"
    };
    std::vector<std::thread> threads;

    for (int i = 0; i < s.size(); i++) {
        threads.push_back(std::thread(print, i, s[i]));
    }

    for (auto &th : threads) {
        th.join();
    }
    return 0;
}
```

Table 13-2. Multithreaded Code

Screenshot	**CABUG:**
	`Printing Integer: 10`
	`Printing String: T.I.P.`
	
	SALCEDO:
	`Printing integer: 10`
	`Printing string: T.I.P.`

Analysis	Thread works by print function separately from the main function thread. When the t1(print, 10, “TIP”) is executed, a new thread named t1 is created. The values 10 and TIP uses them as arguments for the print function. While the main thread continues to the next statement, t1.join() makes it wait until the new thread finishes printing both the integer and the string. The main thread creates another thread to print the given values, and join() ensures that the program waits for that thread to finish before ending.

Table 13-3. Simple One-Threaded Example

Screenshot	**CABUG:**

	3 : Engineers0 : T.I.P. 1 : Competent 2 : Computer 3 : Engineers1 : Competent2 : Computer 0 : T.I.P. **SALCEDO:** 0 : T.I.P.1 : Competent 2 : Computer 3 : Engineers
Analysis	The header files used are #include <iostream>, #include <thread>, #include <vector> and #include <string>. <iostream> is used to output using std::cout, <thread> for creating threads, <vector> to store the strings and <string> to use the string variables. In the void print function, std::to_string(n) was used to convert integers to string, + is added to combine the number, separator and text in a single message. Now to the main function, we initialize the strings which are "T.I.P", "Competent", "Computer", "Engineers", and we use vector to store it by std::vector<std::thread> threads;. We now create a for loop, this for loop creates multiple threads (for (int i = 0; i < s.size(); i++) { threads.push_back(std::thread(print, i, s[i]));. threads.push_back() stores each thread to the vector. The for loop, for (auto &th : threads) { th.join(); } ensures that the program will not exit. th.join(); is used to make the main thread wait until each thread completes its execution. The output is random but still the same since all 4 of the threads are executing every time it runs.

Table 13-4. Multithreaded Example

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

PART A:

QUESTIONS	ANSWERS
1. Write a definition of multithreading and its advantages/disadvantages.	**Cabug:** Multithreading is a technique where a process is divided into smaller execution units called threads that run concurrently. It helps a program manage several activities. https://www.geeksforgeeks.org/operating-systems/multithreading-in-operating-system/ One of the **advantages** is that it improves program responsiveness because users can continue interacting with the application while other tasks are running. It also helps utilize computer resources

	efficiently and can make program execution faster on multicore processors. The **disadvantages** are, it can make programs more difficult to develop and debug because multiple threads may run in different orders. Synchronization problems can also occur when threads access the same data at the same time. In addition, creating and managing too many threads can consume extra memory and processing time. **Salcedo:** It is a programming technique where a single process is divided by two or more threads that can execute at the same time. Each thread performs a specific task within the same process. Advantages: ● Increased performance ● Increased responsiveness ● Resource sharing ● Efficient communication (can exchange data easily) ● Better CPU utilization. Disadvantages: ● Complex programming ● Sync issues ● Difficult to debug ● Resource overhead (as more threads created, the memory increases) ● Deadlock (stuck)
2. Rationalize the use of multithreading by providing at least 3 use-cases.	**Cabug:** The applications of multithreading include web browsers. **Web browsers** use multithreading to handle activities such as loading web pages, playing videos, and responding to user actions, allowing users to continue browsing while content loads.**Online games** use multiple threads to manage player input, sound, artificial intelligence, and other background tasks, helping the game. **Video editing software** uses multithreading to process video frames, apply effects, and handle audio more efficiently. **Salcedo:** These programs use multithreading to perform multiple tasks simultaneously to improve performance, responsiveness, etc.

	• Web-browsers: - Loading webpages - Playing videos - Responding input • Video games: - Graphics - Player input - Game logic • File downloading: - Large file - Multiple files
3. Differentiate between parallelism and concurrency.	**Cabug:** **Concurrency** is when multiple tasks make progress during overlapping periods by taking turns using a processor. **Parallelism** is when multiple tasks execute at the same time using multiple processor cores. Concurrency focuses on managing tasks, while parallelism focuses on performing tasks simultaneously . **Salcedo:** Concurrency happens when two tasks overlap while parallelism can use multiple programs but runs simultaneously on different data.

PART B:

CODE:

partB.cpp > ...

```cpp
#include <iostream>
#include <thread>

int total = 0;

void add(int value) {
    total += value;
}

int main() {
    std::thread T1(add, 1);
    std::thread T2(add, 2);
    std::thread T3(add, 3);

    T1.join();
    std::cout << "After T1.join(): " << total << std::endl;

    T2.join();
    std::cout << "After T2.join(): " << total << std::endl;

    T3.join();
    std::cout << "After T3.join(): " << total << std::endl;

    return 0;
}
```

OUTPUT:

```
After T1.join(): 6
After T2.join(): 6
After T3.join(): 6
```

ANALYSIS:

CABUG:

In this program it uses three threads to add different values to the global variable total. T1 adds 1, T2 adds 2, and T3 adds 3. Since all threads start before the join() functions are called, the other threads can finish their tasks while the program waits for T1. This is why the output can already be 6 after T1.join(). The join() function makes sure that each specified thread has finished before the next statement is executed

SALCEDO:

The program uses three threads to add different values into the global variable. It adds based on their input (T1 = 1 and so on). The threads start before the join() function, the other threads on the other hand while the program itself waits for T1. The output is 6 because the join() function makes each specific thread finish before the next one is executed.

PART C:

CODE:

partC.cpp > ...

```cpp
#include <iostream>
#include <thread>

const int SIZE = 12;
int numbers[SIZE] = {
    18, 7, 29, 14, 3, 27,
    11, 6, 20, 9, 16, 4
};

const int target = 27;
int result = -1;

void linearSearch(int start, int end) {
    for (int i = start; i < end; i++) {
        if (numbers[i] == target) {
            result = i;
            return;
        }
    }
}

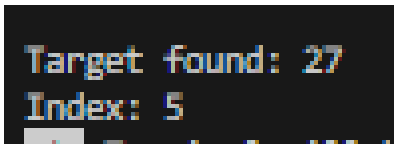
int main() {
    std::thread T1(linearSearch, 0, 4);
    std::thread T2(linearSearch, 4, 8);
    std::thread T3(linearSearch, 8, 12);

    T1.join();
    T2.join();
    T3.join();

    if (result != -1) {
        std::cout << "Target found: " << target << std::endl;
        std::cout << "Index: " << result << std::endl;
    } else {
        std::cout << "Target not found." << std::endl;
    }

    return 0;
}
```

OUTPUT:



ANALYSIS:

CABUG:

The array is divided into three sections, and each thread searches four elements. In the output, the target value 25 is found at index 5 because it is the sixth element in the array, and indexing starts at 0. The join() function ensures that all three threads finish searching before the result is displayed. Since the array uses fixed numbers, the output stays the same every time the program runs.

SALCEDO:

The array is divided into three sections. Each thread is searching for four elements (0,4,8,12). The join() function makes all three threads finish searching before the result is displayed. The array is fixed so the output remains the same (Output is 27).

Conclusion:

CABUG:

In this activity, we learned how multithreading works and how it allows a program to perform different tasks using multiple threads. I learned how to create threads using thread and use join() to make sure that each thread finishes its task. In the supplementary activity, I get the idea how multithreading can be applied to adding values to a global variable and searching for a target in an array. The order of thread execution can affect the output, especially when multiple threads access the same variable. I think I did well in completing the activity, but I still need to improve my understanding of how to manage threads properly and avoid unexpected results when using multithreading.

SALCEDO:

We managed to finish the Hands-on Activity. This activity helps us understand how multithreading works, how it allows the program to work using different threads. In the procedure part, it shows how it works and it's always the same output but different position. In the supplementary activity, we were able to show how multithreading can apply on searching a target. Overall, I think we did good and understand how multithreading works.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

COMPUTER ENGINEERING

QUEZON CITY

Purpose of Use: **Idea generation and code analysis**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **We used ChatGPT to generate ideas as well as code analysis but the sentences used were ours only.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.